

AI Audit Report

Student: Nguyen Phan Hung Linh Student ID: 23127081 Session date: 2026-08-09 Timezone: Asia/Ho_Chi_Minh (UTC+07:00)

Declaration

I use AI tools for the following tasks:

- OpenAI Codex collaborating agent: source analysis, test-case suggestions, initial script drafts, and critique of its own drafts.

I reviewed the requirements and runtime results, corrected the AI drafts, and accepted responsibility for the final tests. The collaborating agent did not edit the repository or generate the execution evidence.

The collaboration interface did not expose a separate wall-clock timestamp on each agent message. The times below are recorded from the session order and nearby Git commits; they are marked approximate instead of being presented as exact platform metadata. Exact report timestamps and Git commit timestamps remain in the repository.

Conversation with the collaborating AI

A1 - Source-grounded feature analysis

- Tool: OpenAI Codex collaborating agent
- Time: 2026-08-09, approximately 15:43 ICT
- My prompt:

Act as the external AI assistant that a third-year software engineering student is guiding for HW04. Do not edit files. First interaction only: inspect the local eshop-sut code and identify the exact user-visible behavior, routes, prerequisites, and likely automation risks for FR-03 (forgot/reset password), FR-09 (discount coupons), and FR-12 (admin access control). Return concise findings with source file references. Do not generate scripts yet; this staged workflow will be recorded in the AI audit.

- AI output: Identified the web/API routes, credentials, fixture requirements, and expected risks. It found the 4-digit OTP, missing reset controls, whitespace-based password regex, incorrect percent formula, > threshold, unauthenticated coupon application, trusted body user_id, middleware without role checking, and public product mutations.
- Human review: I checked these findings against server.js, ForgotPassword.jsx, Checkout.jsx, and the admin application before requesting test cases.

A2 - Design exactly 36 cases

- Tool: OpenAI Codex collaborating agent
- Time: 2026-08-09, approximately 15:50 ICT
- My prompt:

Second interaction. Based on your source-grounded analysis, design exactly 12 independent, data-driven cases for each feature FR-03, FR-09, and FR-12 (36 total). For every case give: stable ID, type (positive/negative/edge/security), layer (UI/API), input data, expected result from the written requirements (not the buggy implementation), and whether it likely exposes a current SUT defect. Keep destructive cases minimal and identify setup/cleanup. Also propose separate JSON field schemas for the three datasets. Do not write Playwright code yet.

- AI output: Produced 12 cases for each feature with IDs, layers, expected results, setup/cleanup, defect predictions, and JSON schemas. It covered OTP and password classes, coupon arithmetic/boundaries/authentication/usage, and admin UI/API access control.

- Human review: I kept the 36-case structure but later replaced the impossible inactive-coupon fixture with an empty-code case and separated mixed-purpose expectations.

A3 - Initial FR-03 draft

- Tool: OpenAI Codex collaborating agent
- Time: 2026-08-09, approximately 15:55 ICT
- My prompt:

Third interaction: generate an initial Playwright JavaScript draft for FR-03 only, using the 12-case design and a separate JSON file. Prefer accessible locators and explicit assertions. Tests must express the written requirements, so tests may fail when they reveal SUT defects. Keep tests independent and avoid modifying the SUT. Include comments only where setup or an expected defect is not obvious. Return code blocks for `test-data/fr03-cases.json` and `tests/fr03.spec.js`; do not edit files.

- AI output: Returned a 12-row JSON dataset and a Playwright draft with unique account setup, OTP extraction, weak-password cases, cross-email OTP testing, login verification, and cleanup.
- Mistakes found: It used CommonJS in an ESM repository, mixed step assertions into the OTP case, used positional locators, and depended too heavily on alert text.
- Human changes: Converted to ESM, isolated primary assertions, reduced alert coupling, added deterministic fixture counters, and executed the cases before proceeding.

A4 - Initial FR-09 draft

- Tool: OpenAI Codex collaborating agent
- Time: 2026-08-09, approximately 16:03 ICT
- My prompt:

Fourth interaction: generate an initial Playwright JavaScript draft for FR-09 only, using the agreed 12 cases and a separate JSON file. Use the checkout UI for the UI-labeled cases and Playwright request for API cases. Express requirement-correct expectations even when the SUT should fail. Avoid raw SQLite access; create users and usage fixtures through available APIs where possible. Return code blocks only; do not edit files.

- AI output: Returned coupon JSON and Playwright code for arithmetic, expired/unknown/minimum cases, guest behavior, usage fixtures, per-user isolation, and identity spoofing.
- Mistakes found: The guest locator matched the navbar, deleting a coupon did not simulate inactivity, ordinary calls sent a client-controlled user ID, and the isolation case also asserted broken percent math.
- Human changes: Replaced the broad guest lookup with URL state, replaced the fake inactive case with empty-code coverage, omitted `user_id` from normal requests, and made per-user isolation its own assertion.

A5 - Initial FR-12 draft

- Tool: OpenAI Codex collaborating agent
- Time: 2026-08-09, approximately 16:08 ICT
- My prompt:

Fifth interaction: generate an initial Playwright JavaScript draft for FR-12 only, using the agreed 12 cases and a separate JSON file. Include both admin UI login behavior and backend access-control checks. Use unique mutation fixtures and cleanup only what each test creates. Assert requirement-correct 401/403 responses even though the SUT may wrongly return success. Return code blocks only; do not edit files.

- AI output: Returned admin-login tests and API cases for admin, guest, invalid-token, and normal-user actors. Mutation drafts captured created IDs for cleanup.
- Mistakes found: It always supplied a request body option, relied mainly on status codes, and tied the normal-user UI case to exact alert wording.

- Human changes: Built request options conditionally and added database-visible state checks after unauthorized coupon, product, and category attempts.

A6 - Critique after real Chromium execution

- Tool: OpenAI Codex collaborating agent
- Time: 2026-08-09, approximately 16:14 ICT
- My prompt:

Sixth interaction, human-review feedback. I adapted your CommonJS drafts to ESM and ran all 12 cases per feature in Chromium. Results: FR-03 7 passed/5 failed; FR-09 7 passed/5 failed; FR-12 5 passed/7 failed. I also found your FR09 guest poll falsely matched the navbar text “Đăng nhập”, and the inactive-coupon case cannot establish `is_active=0` through available APIs. Critique your drafts: list the mistakes or fragile choices you made, why you made them, and concrete fixes I should apply before the three-browser matrix. Keep this concise and do not edit files.

- AI output: Acknowledged the ESM mismatch, combined assertions, fragile positional locators, alert dependence, false guest match, invalid inactive setup, direct `user_id`, orphaned usage rows, unconditional request data, and missing state verification. It recommended a freshly seeded matrix.
- Human review: I applied the relevant corrections, restarted the backend to reseed the database, and then ran the nine official feature-browser reports.

A7 - Final audit summary

- Tool: OpenAI Codex collaborating agent
- Time: 2026-08-09, approximately 16:30 ICT
- My prompt:

Seventh and final interaction. Produce a concise chronological audit summary of our six earlier interactions. For each, include the exact purpose, the main output you provided, and the mistakes later identified. Do not claim that your code was used unchanged. End with a short statement distinguishing your contribution from the student's human review and execution evidence. Do not edit files.

- AI output: Summarized the six stages and explicitly stated: its contribution was source analysis, test-design assistance, and initial draft code; the student performed the review, adapted the drafts, executed tests, found draft-level problems, and produced runtime evidence.

Final responsibility statement

The final files are not raw AI output. I used the AI drafts as reviewable input. The submitted assertions follow the written requirements, and the HTML reports, screenshots, traces, Git history, and GitHub Issues come from actual execution or repository operations performed during this session.